

**Автономная некоммерческая организация высшего образования
«Университет Иннополис»**

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 «Информатика и вычислительная техника»**

**FINAL QUALIFICATION WORK
(BACHELOR'S THESIS)
Academic Program
09.03.01 «Computer Science»**

**Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»**

**Field of Study:
«Computer Science»**

Тема

**Среда выполнения OCI, ориентированная на безопасность:
проектирование и сравнительная оценка с использованием
изолированных сред выполнения**

Topic

**Security-First OCI Runtime: Design and Comparative Evaluation
with Sandboxed Runtimes**

Работу выполнил /
Prepared by

Платонов Иван Васильевич

подпись / signature

Руководитель
выпускной
квалификационной
работы / *Final
Qualification Work
Supervisor*

**Садовых Андрей
Александрович**

подпись / signature

Консультанты
Consultants

**Салтанов Кирилл
Сергеевич**

подпись / signature

Иннополис, Innopolis, 2026

Оглавление

1 Введение

Выпускная квалификационная работа посвящена проектированию, реализации и оценке среды выполнения контейнеров OCI, ориентированной на безопасность. Актуальность темы определяется тем, что контейнеры разделяют ядро хоста, тогда как эталонный `runc` ориентирован на производительность, а не на изоляцию **wang2022, volpert2024**. `Seccomp`, `AppArmor` и `capabilities` требуют ручной настройки; песочницы `gVisor` и `Kata` дают сильную изоляцию ценой 20–100% накладных расходов **viktorsson2020**; кластерные рекордеры профилей предполагают инфраструктуру оркестрации. Операторы `plain runc` остаются без интегрированного инструмента автоматизации.

Цель — спроектировать и экспериментально оценить форк `runc`, генерирующий профили `seccomp`, `AppArmor` и `capabilities` из наблюдаемого поведения нагрузки одним флагом CLI. Объектом исследования является изоляция контейнерных процессов на Linux в рамках OCI. Предметом — интегрированная runtime-локальная генерация профилей `confinement`. Научная новизна заключается в объединении трёх механизмов ядра в двухфазный конвейер «сканирование — принудительное исполнение» с `cgroup-wide` трассировкой `capabilities` и самовыполняемыми OCI-хуками внутри бинарного файла `runtime`.

2 Краткое содержание

В основной части выполнен обзор литературы по контейнерной безопасности, изолированным runtime и автоматической генерации `seccomp` **lopes2020**. Сравнения runtime, как правило, противопоставляют `runc` с политиками по умолчанию песочницам, не измеряя потенциал `hardened shared-kernel runtime`;

автоматизация чаще охватывает только seccomp, а AppArmor и capabilities остаются фрагментированными **mattetti2020**.

Проектная часть формулирует требования к сканеру: однофлаговый UX, три профиля за один проход, совместимость с upstream runc, минимальные зависимости и финализация, которая никогда не расширяет привилегии. Модель угроз охватывает эскалацию через неиспользуемые syscalls и capabilities, выход за пределы файловой системы и post-exploitation после компрометации приложения; уязвимости прикладного уровня и supply-chain атаки вынесены за рамки.

Независимо от режима сканирования форк меняет дефолт безопасности: runc spec теперь генерирует пустой набор capabilities, а исторические наборы upstream (3 capabilities) и Docker (14 capabilities) доступны через два взаимоисключающих флага. Архитектура реализует режим `-security-scan`: ослабление confinement в памяти, установка хуков, запуск контейнера, остановка трассировщиков, сужение config.json. Seccomp записывается через runtime-трассировку syscalls; capabilities — через eBPF-трассировку по cgroup v2; AppArmor обучается в complain mode. Хуки реализованы подкомандами runc; OCI bundle не получает исполняемых артефактов.

Реализация — форк opencontainers/runc. Лабораторный стенд runc-hardened-test на Ubuntu 24.04 сравнивает stock runc и hardened runtime на уязвимом Flask-приложении: baseline, scan с легитимным трафиком, enforcement и повтор post-exploitation payload. Один проход `-security-scan` формирует seccomp.json, профиль AppArmor и журнал capabilities. После scan-then-enforce reconnaissance, чтение `/etc/shadow`, reverse shell и mount probe блокируются enforced seccomp; ping и upload календаря сохраняются после документированных корректировок. Ограничения: покрытие равно пройденным путям; file

capabilities не попадают в trace; сравнение с gVisor и Kata носит аналитический характер по данным wang2022, volpert2024.

3 Заключение

В результате работы разработан и экспериментально оценён security-first форк OCI runtime runc. Форк вносит три изменения в слой runtime: пустой набор capabilities по умолчанию с двумя opt-in флагами, флаг `–security-scan` для генерации профилей seccomp, AppArmor и capabilities, и сужение `config.json` до наблюдаемого набора привилегий без его расширения. Основной вклад — интегрированная генерация профилей seccomp, AppArmor и capabilities в воспроизводимом runtime-локальном конвейере с одним флагом `–security-scan`. Полученные результаты подтверждают существенное улучшение post-exploitation containment относительно stock runc на тестируемой нагрузке при сохранении легитимного поведения приложения. Выводы ограничены одним web-workload и качественными метриками безопасности; дальнейшая работа включает слияние file capabilities, расширение лаборатории и количественную оценку производительности относительно изолированных runtime reeves2021runtime.

4 Список использованной литературы

1. Wang X., Johannesson D. Performance and isolation analysis of RunC, gVisor and Kata Containers runtimes // Cluster Computing. 2022. Vol. 25. P. 2363–2380.
2. Volpert S., Winkelhofer S., Wesner S., Domaschka J. An Empirical Analysis of Common OCI Runtimes' Performance Isolation Capabilities // Proceedings of the 15th ACM/SPEC ICPE. 2024. P. 60–70.

3. Viktorsson W., Klein C., Tordsson J. Security-Performance Trade-offs of Kubernetes Container Runtimes // 2020 IEEE MASCOTS. 2020. P. 1–4.
4. Lopes N., Martins R., Correia M. E., Serrano S., Nunes F. Container Hardening Through Automated Seccomp Profiling // Proceedings of the 6th International Workshop on Container Technologies and Container Clouds. 2020. P. 31–36.
5. Mattetti M., Allouche Y. Automatic security hardening and protection of linux containers // Computers & Security. 2020. Vol. 96. Article 101921.
6. Reeves M., Tian D. J., Bianchi A., Celik Z. B. Towards Improving Container Security by Preventing Runtime Escapes // 2021 IEEE SecDev. 2021. P. 38–46.